

Chapter 1

1. Introduction to Image Datasets

Images are represented as matrices of pixel values. Machine learning models use these pixel values as features for classification tasks. TensorFlow provides built-in datasets like CIFAR-10, which are excellent for learning.

- What do you think the dimensions of an image dataset might look like? How would the shape differ for grayscale and color images?

2. Loading an Image Dataset

In this step, you will load the CIFAR-10 dataset using TensorFlow.

Tasks:

1. Find the TensorFlow function used to load CIFAR-10. Use the TensorFlow/Keras documentation if needed.
2. Load the dataset into training and test sets. What are the shapes of the images and labels?
3. How many images are there in the training and test sets? How many classes does CIFAR-10 have?

Questions to Explore:

- How are the labels stored? Are they numbers, one-hot encoded vectors, or something else?
- What do these numbers represent?

3. Visualizing Images

Visualization helps you understand the dataset better. In this step, you will plot several images from the CIFAR-10 dataset along with their labels.

Tasks:

1. Display a few random images from the training dataset. Can you write a loop to display 10 images in a grid?
2. CIFAR-10 has 10 classes. Try to match the numeric labels to the class names. Can you find a list of class names in the CIFAR-10 documentation?

Questions to Explore:

- Do the images seem evenly distributed across the classes?
- What do you notice about the resolution and quality of the images? How might this affect model training?

4. Data Preprocessing

Neural networks work best when the data is normalized. The pixel values in the CIFAR-10 dataset range from 0 to 255. You need to scale these values to be between 0 and 1.

Tasks:

1. Write a function to normalize the images. Can you figure out how to divide all pixel values by 255? Why is normalization important?
2. Verify that your normalization is correct. What are the maximum and minimum pixel values after normalization?

Questions to Explore:

- How would training be affected if the data wasn't normalized?
- Are there other preprocessing steps you think might be useful for this dataset?

Chapter 2

Splitting Data for Model Training and Evaluation

Objective: To learn the importance of splitting data into training, validation, and test sets and to perform the split. This chapter will guide you through understanding why these splits are necessary and how they contribute to building a robust machine learning model.

1. Why Do We Split Data?

When building a machine learning model, it is crucial to evaluate how well the model generalizes to unseen data. Splitting the dataset into training, validation, and test sets serves this purpose.

Tasks:

1. Research and write down the purpose of each of the following sets:
 - **Training Set:** Why do we need this, and how does it contribute to learning?
 - **Validation Set:** Why do we need a separate validation set? What role does it play in model selection and hyperparameter tuning?
 - **Test Set:** When should this set be used? Why is it important not to use it during training or validation?
2. Think about the challenges that might arise if the data is not split properly (e.g., if the test set overlaps with the training set). How might this affect model evaluation?

3. Choosing Split Ratios

Common split ratios for datasets include:

- **Training Set:** 70–80% of the data.
- **Validation Set:** 10–20% of the data.
- **Test Set:** 10–20% of the data.

Tasks:

1. What factors influence the choice of split ratios? Consider the dataset size and complexity.
2. What might happen if the validation set is too small or too large?
3. Decide on the split ratios you would like to use for the CIFAR-10 dataset and justify your choice.

4. Shuffling the Data

Shuffling ensures that the data is randomly distributed, preventing patterns in the order of the data from influencing the splits.

Tasks:

1. Why is shuffling important before splitting the data?
2. Imagine that the CIFAR-10 dataset is sorted by class (e.g., all airplane images appear first, followed by all automobile images). What would happen if you split the dataset without shuffling?
3. Find out how to shuffle data in TensorFlow or another library you are using.

Chapter 3

Building a Simple Fully Connected Neural Network

Objective:

To implement a simple fully connected neural network (FCNN) using TensorFlow and Keras for classifying images in the CIFAR-10 dataset. This session will guide you in defining the architecture, setting up training, and running the model.

1. Define a Simple Neural Network Architecture

Tasks:

1. Use **Keras Sequential API** to define the structure of your fully connected neural network.
2. Your model should include:
 - **Input Layer:** This layer will accept flattened images (32x32x3 reshaped to a vector of 3072).
 - **Hidden Layers:** Add 1–2 dense (fully connected) layers with ReLU activation.
 - **Output Layer:** Add a dense layer with 10 neurons (for 10 classes) and softmax activation.
3. Ensure you include Flatten() as the first layer to convert 3D image arrays into 1D vectors.

Questions to Explore:

- Why does the input layer require a Flatten() operation for image data?
- What does the softmax activation function do in the output layer, and why is it important for classification?

2. Compile the Model

Tasks:

1. Compile your model using the following:
 - **Loss Function:** Use sparse_categorical_crossentropy for multi-class classification.
 - **Optimizer:** Use Adam or experiment with SGD.
 - **Metrics:** Include accuracy as a metric to evaluate performance during training.
2. What does compiling a model mean in TensorFlow? Write a short explanation in your notes.

3. Train the Model

Tasks:

1. Use the **model.fit()** function to train the FCNN on the training dataset.
 - Pass the training images and labels, and specify the batch size and number of epochs.
 - Use your validation set to monitor performance during training.
2. Plot the training and validation accuracy over epochs. What do you observe? Is the model overfitting or underfitting?

4. Evaluate the Model

Tasks:

1. Evaluate your trained model on the test set using `model.evaluate()`.
2. Report the test accuracy. How does it compare to the training and validation accuracy?
3. Identify possible reasons if the test accuracy is significantly lower than the training accuracy. What might be done to improve it?

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense
from tensorflow.keras.optimizers import Adam

# 1. Load the CIFAR-10 dataset
# Use the dataset you prepared in Chapter 2 (train, validation, and test splits)

# 2. Define the model architecture
model = Sequential([
    Flatten(input_shape=(32, 32, 3)), # Flatten the input image
    Dense(128, activation='relu'), # Hidden layer with 128 neurons
    Dense(64, activation='relu'), # Optional second hidden layer
    Dense(10, activation='softmax') # Output layer with 10 classes
])

# 3. Compile the model
model.compile(
    optimizer=Adam(learning_rate=0.001), # Experiment with learning rates
    loss='sparse_categorical_crossentropy', # For integer labels
    metrics=['accuracy']
)

# 4. Train the model
history = model.fit(
    train_images, train_labels, # Training data
    validation_data=(val_images, val_labels), # Validation data
    epochs=20, # Number of training epochs
    batch_size=64 # Batch size
)
```

5. Plot training and validation accuracy

```
import matplotlib.pyplot as plt
```

```
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.title('Training and Validation Accuracy')
plt.show()
```

6. Evaluate the model on the test set

```
test_loss, test_accuracy = model.evaluate(test_images, test_labels)
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")
```

Reflection Questions:

1. How did you decide on the number of layers and neurons in your FCNN?
2. What observations did you make from the training and validation accuracy plots?
3. How does the test accuracy compare to the training and validation accuracy? What might explain these differences?
4. What changes would you consider making to improve the performance of your FCNN?

